Due: Thursday, February 12, 2026 11:59pm

Submit on GL.

---

1 Penultimate Item

Making good use of Haskell library functions for lists, write a function `secondLast` with type:

```
secondLast :: [a] -> a
```

that returns the second last item in a list. If the list has fewer than two items then use the `error` function to report a problem.

For example, running your code in `ghci` should produce:

```
ghci> secondLast [2, 3, 9, 4, 8]
4
ghci> secondLast "penultimate"
't'
ghci> secondLast [1.3]
*** Exception: List must have at least two items
CallStack (from HasCallStack):
  error, called at secondLast.hs:2:20 in main:Main
ghci>
```

Bonus points: 10% bonus for *not* using the `!!` operator on lists.

Place your `secondLast` function in a file named `hw02_1.hs` and export the function in the module `HW02_1`. There should not be a `main` function in this file. Include your name and userid in comments as well. The top of your file should look something like:

```
module HW02_1 where
-- Julius Nepos (emprr474)
```

Your `hw02_1.hs` file should compile with `main02_1.hs` using `ghc`:

```
ghc main02_1.hs hw02_1.hs -o main02_1.out
```

Then, you can run the program with and your output should look like:

```
linux6% ./main02_1.out
17
't'
5.6
linux6%
```

You can find `main02_1.hs` on GL under

```
/afs/umbc.edu/users/c/h/chang/pub/cs331stuff/hw02/
```

Submit your `hw02_1.hs` file on GL as before.

2 Middle Half

Making good use of Haskell library functions for lists, write a function

```
middleHalf :: [a] -> [a]
```

that returns a portion of the list that includes the items in the second and third quarter — i.e., the middle half. If the length of the list is not divisible by 4, then we round down both the starting index and the length of the result. For example, running your code in `ghci` should produce:

```
ghci> middleHalf [1,2,3,4,5,6,7,8,9,10,11,12]
[4,5,6,7,8,9]

ghci> middleHalf [1..32]
[9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24]

ghci> middleHalf [1..31]
[8,9,10,11,12,13,14,15,16,17,18,19,20,21,22]

ghci> middleHalf ['a'..'z']
"ghijklmnopqrs"
```

Place your definition of the `middleHalf` function in a file named `hw02_2.hs` and export the function in the module `HW02_2`. There should not be a `main` function in this file. Include your name and userid in comments as well. The top of your file should look something like:

```
module HW02_2 where
-- Mary A. Evans (geliot2)
```

Your `hw02_2.hs` file should compile with `main02_2.hs` using `ghc`:

```
ghc main02_2.hs hw02_2.hs -o main02_2.out
```

Then, you can run the program and your output should look like:

```
linux6% ./main02_2.out
[7,8,9,10,11,12,13,14,15,16,17,18]
" which cannot look at a subject from va"
"GHIJKLMNOPQRS"
linux6%
```

You can find `main02_2.hs` on GL under

```
/afs/umbc.edu/users/c/h/chang/pub/cs331stuff/hw02/
```

Submit your `hw02_2.hs` file on GL as before.

3 Three Little Piggies

We want a Haskell function that given a `String` parameter returns `True` if the parameter's first or second character is an `'o'`. If the parameter is an empty string, then the function should return `False`. If the parameter has only one character, then the function should return `True` if and only if that character is `'o'`.

- Define a function `piggy1` (as described above) using conditional expressions.
- Define a function `piggy2` (which does the same thing) using guarded equations.
- Define a function `piggy3` (which again does the same thing) using pattern matching.

The functions must be named `piggy1`, `piggy2` and `piggy3`. Each function must have type

```
String -> Bool
```

Using `ghci` we should get as output:

```
ghci> piggy1 "object"
True
ghci> piggy1 "class"
False
ghci> piggy1 "constant"
True
```

Place all 3 functions in a file named `hw02_3.hs` and export these functions in the module `HW02_3`. Include your name and userid as well. Submit your `hw02_3.hs` file on GL as usual. The top of your file should look like:

```
module HW02_3 where
-- Joseph Jacobs (ftales3)
```

Your `hw02_3.hs` file should compile with `main02_3.hs` using `ghc`:

```
ghc main02_3.hs hw02_3.hs -o main02_3.out
```

Then, you can run the program and your output should look like:

```
linux6% ./main02_3.out

Testing piggy1:
[False,True,True,False,False,True,False,False,True,True,False]

Testing piggy2:
[False,True,True,False,False,True,False,False,True,True,False]

Testing piggy3:
[False,True,True,False,False,True,False,False,True,True,False]

linux6%
```

You can find `main02_3.hs` on GL under

```
/afs/umbc.edu/users/c/h/chang/pub/cs331stuff/hw02/
```